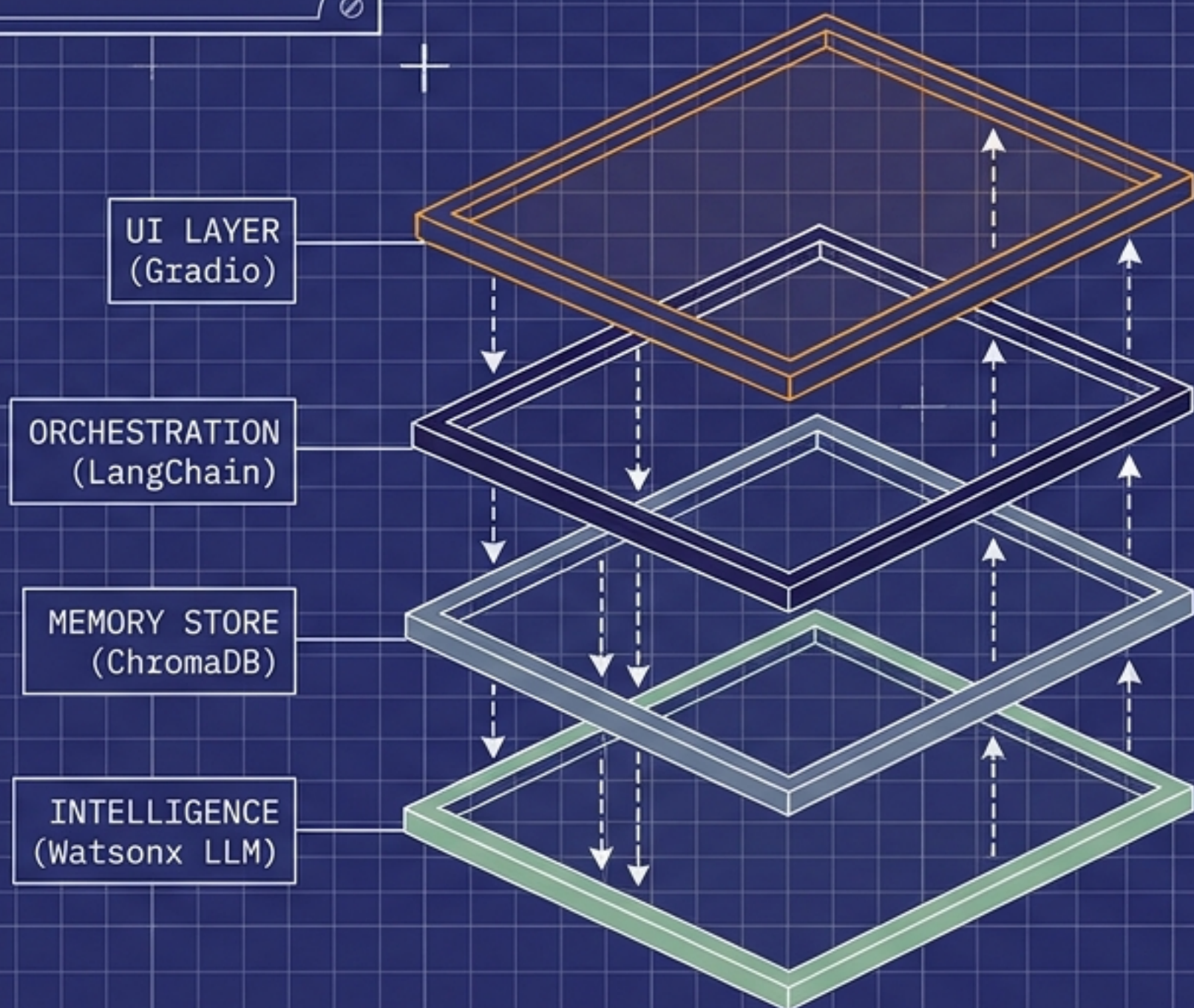


BUILDING MODERN AI: THE RAG APPLICATION BLUEPRINT

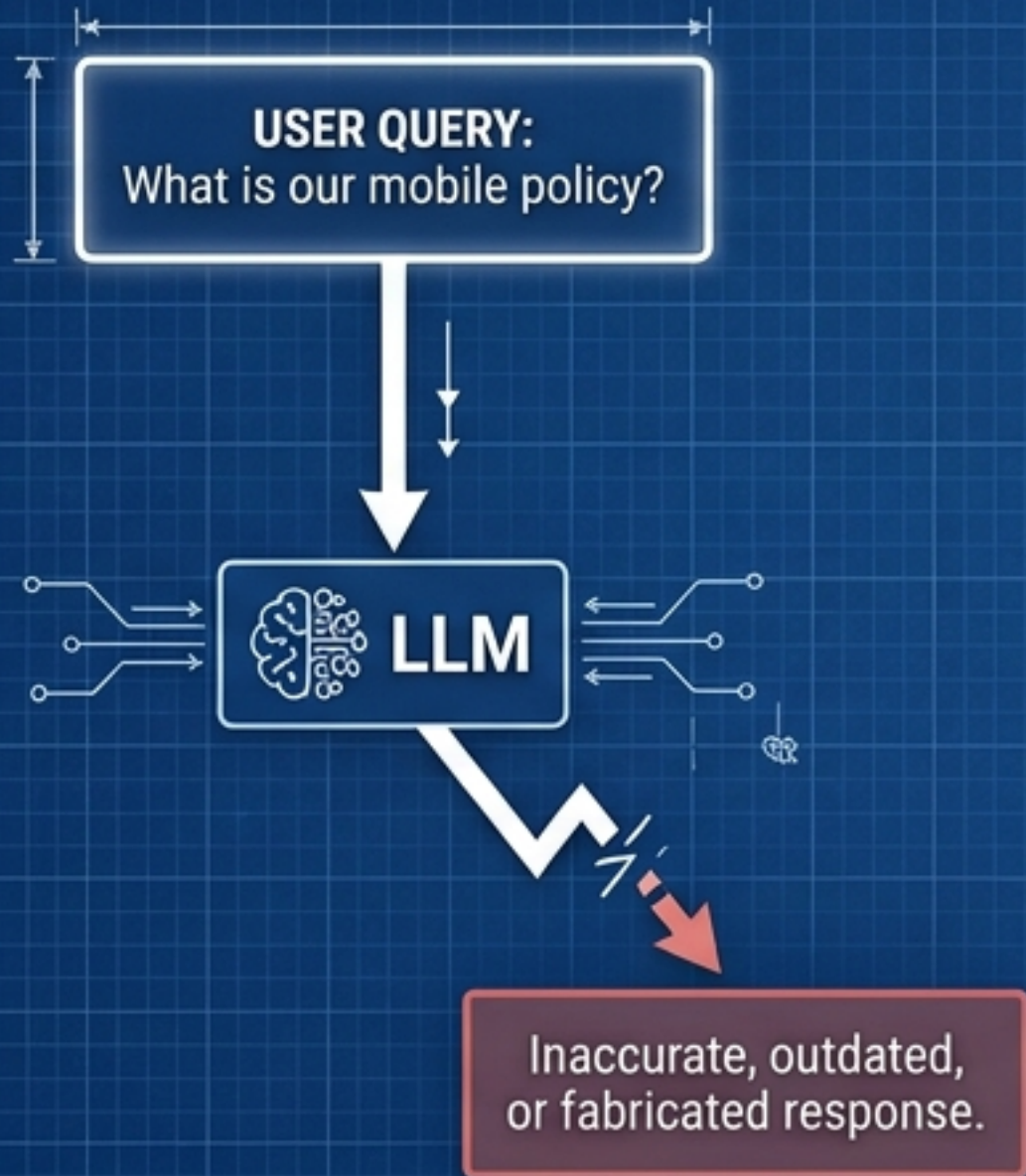
- * A visual guide to assembling Retrieval-Augmented Generation systems.
- * From vector physics to full-stack deployment using LangChain, Watsonx, and Gradio.



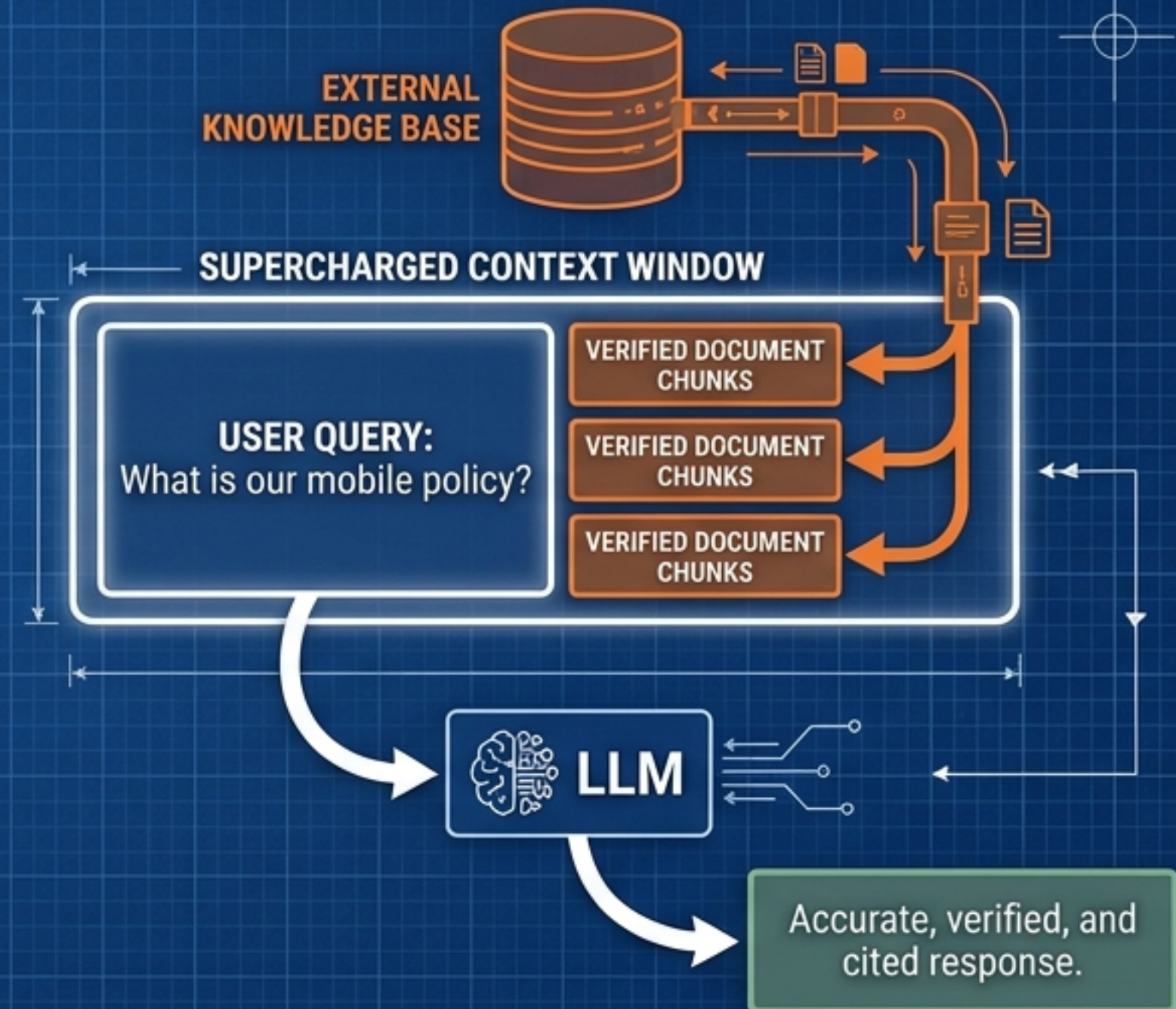
STANDARD LARGE LANGUAGE MODELS POSSESS STATIC, ISOLATED INTELLIGENCE

Pre-trained models excel at general tasks but fail on domain-specific, confidential, or evolving data (e.g., a company's internal mobile policy). RAG fixes this by injecting authoritative, up-to-date data directly into the context window at runtime, eliminating the need for expensive model retraining.

STANDARD LLM ARCHITECTURE



RAG SYSTEM ARCHITECTURE

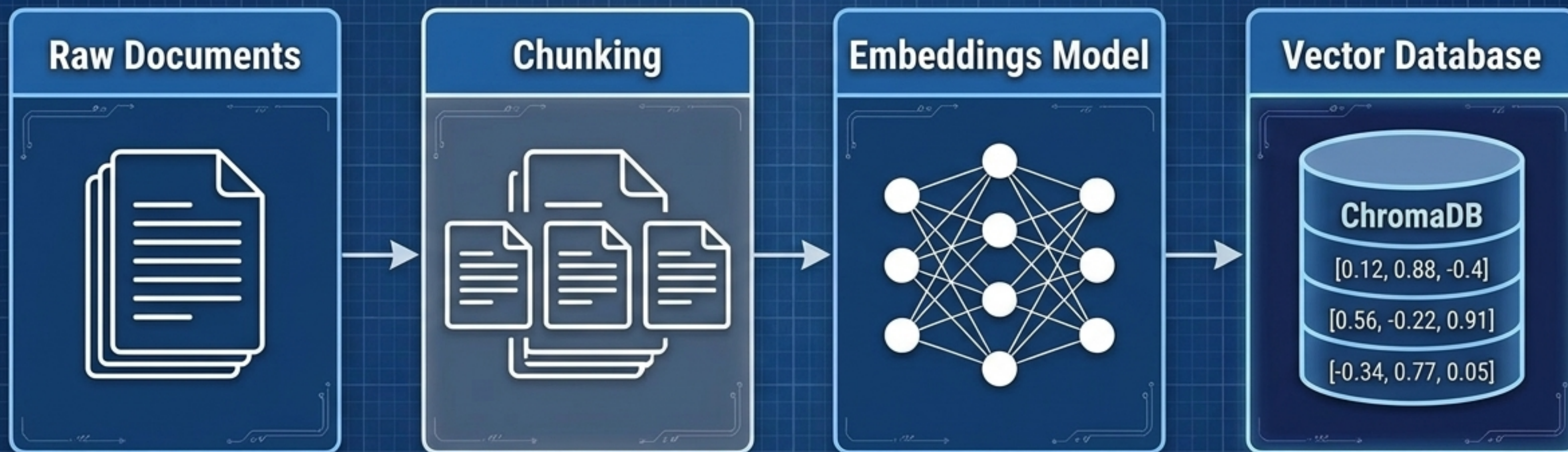


The RAG advantage over standalone generative models

Capability	Standard LLM	RAG System
Handling Outdated Info	Static; limited strictly to training cutoff date.	Dynamic; updates instantly as external databases are refreshed.
Hallucination Risk	High; prone to fabricating facts to fill knowledge gaps.	Low; mathematically grounded in retrieved text chunks .
Source Verification	Impossible; sources are lost in neural network weights.	Built-in; exact document chunks and page numbers are cited .
Context Constraints	Requires stuffing all context manually, increasing cost/time.	Selectively retrieves only the most relevant mathematical chunks .

Phase 1: Transforming static documents into searchable intelligence

Large documents cannot fit entirely into an LLM. They must be split into smaller, manageable chunks. These chunks are transformed into vectors and indexed in a specialized database designed for rapid similarity search.



The anatomy of an embedding maps language to mathematics

Embeddings use tokenization to split text into sub-words. A pre-trained embedding model transforms these IDs into high-dimensional vectors, capturing the deep semantic meaning of the text chunk.

Step 1: Raw Text

Company Mobile Policy

Step 2: Tokenization

[Company]

ID: 4092

[Mobile]

ID: 1104

[Policy]

ID: 8821

Step 3: Neural Encoder



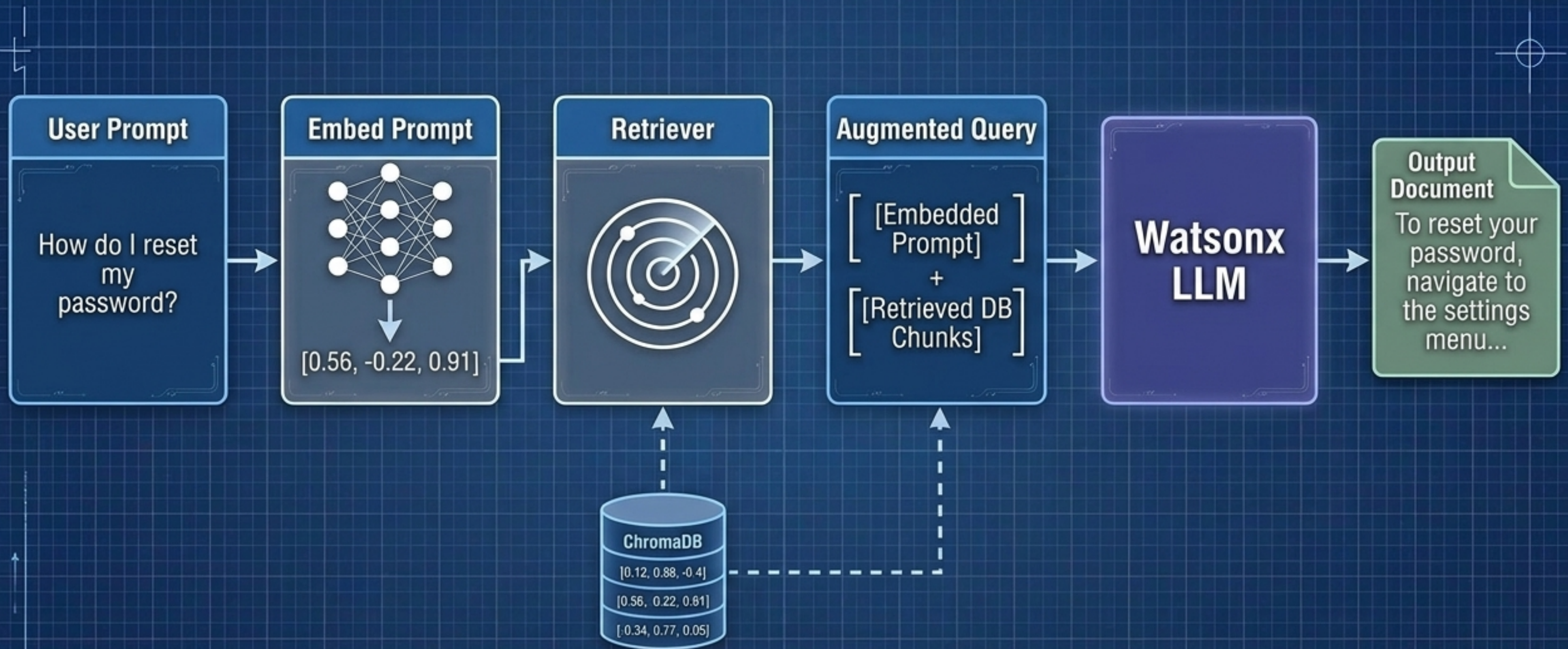
ibm/slate-125m-english

Step 4: Vectorization

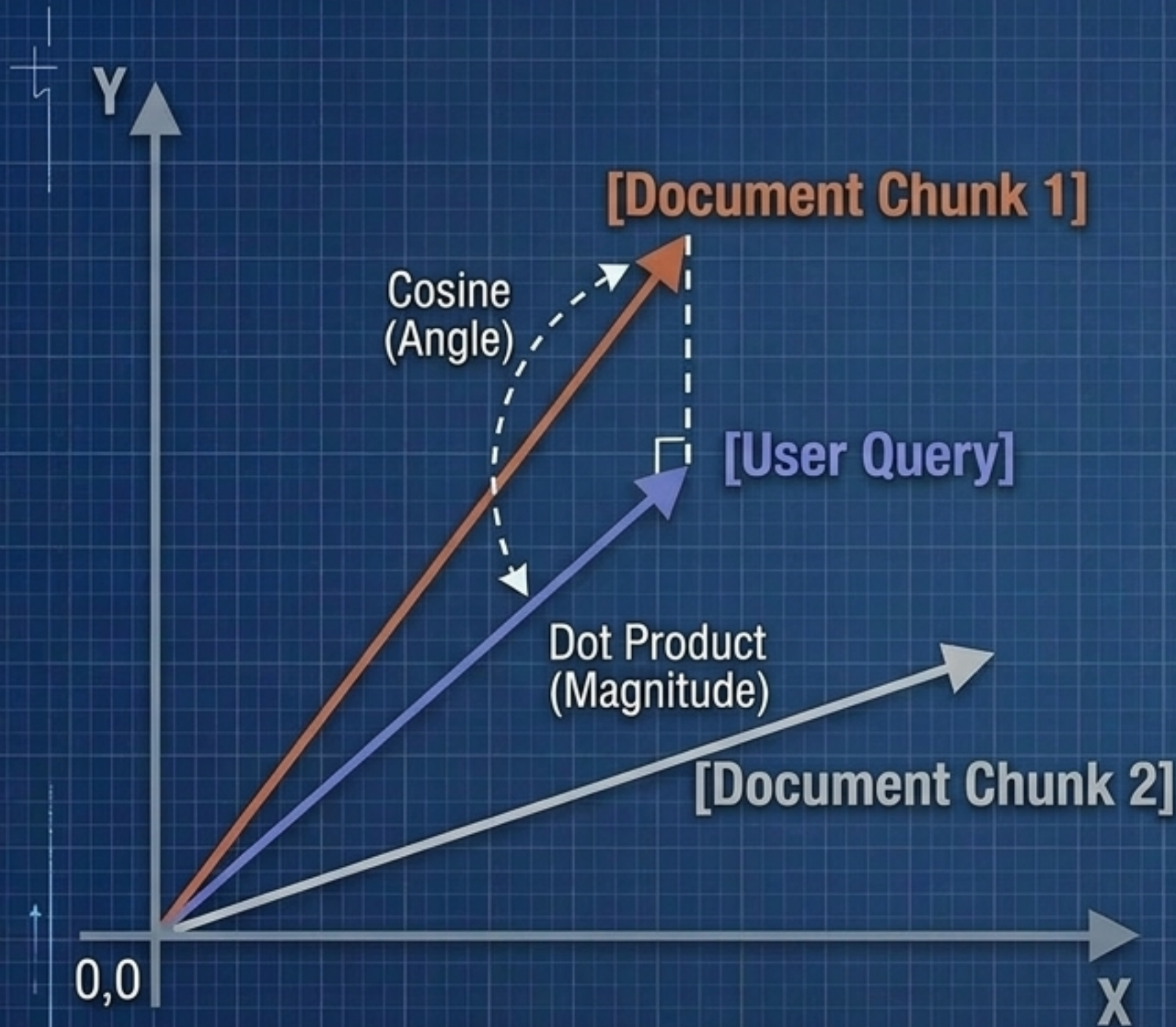
[0.124, -0.451, 0.892, 0.331, -0.901 ... 384 dimensions]

Phase 2: Similarity search and augmented generation

The system embeds the user's prompt using the identical encoder used for the documents. The Retriever calculates the mathematical distance between the prompt vector and context vectors to pull the Top K most relevant chunks.



Retrieval physics relies on angular distance and magnitude



Cosine Similarity

Measures the angular difference between vectors, ignoring size. Best used when finding documents with the closest thematic direction or semantic meaning is critical.

Dot Product

Considers both the direction (angle) and the magnitude (length) of the vectors. Best used when prioritizing overall alignment and giving weight to longer or more robustly matched documents.

Gradio wraps complex Python functions in interactive web interfaces

Gradio is an open-source Python library that transforms backend machine learning models into user-friendly, shareable web applications. No JavaScript, CSS, or web hosting experience is required.

Backend Code

```
import gradio as gr

demo = gr.Interface(
    fn=retriever_qa,
    inputs=[gr.File(), gr.Textbox()],
    outputs=gr.Textbox()
)
demo.launch()
```

Frontend Web App

1
Drop File Here
(Upload PDF)

2
Input Query

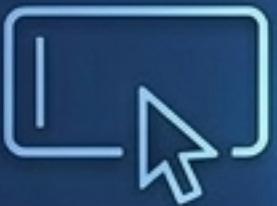
Enter your question here...

Submit

Output

3
This is the retrieved answer based
on your query and the uploaded
document.

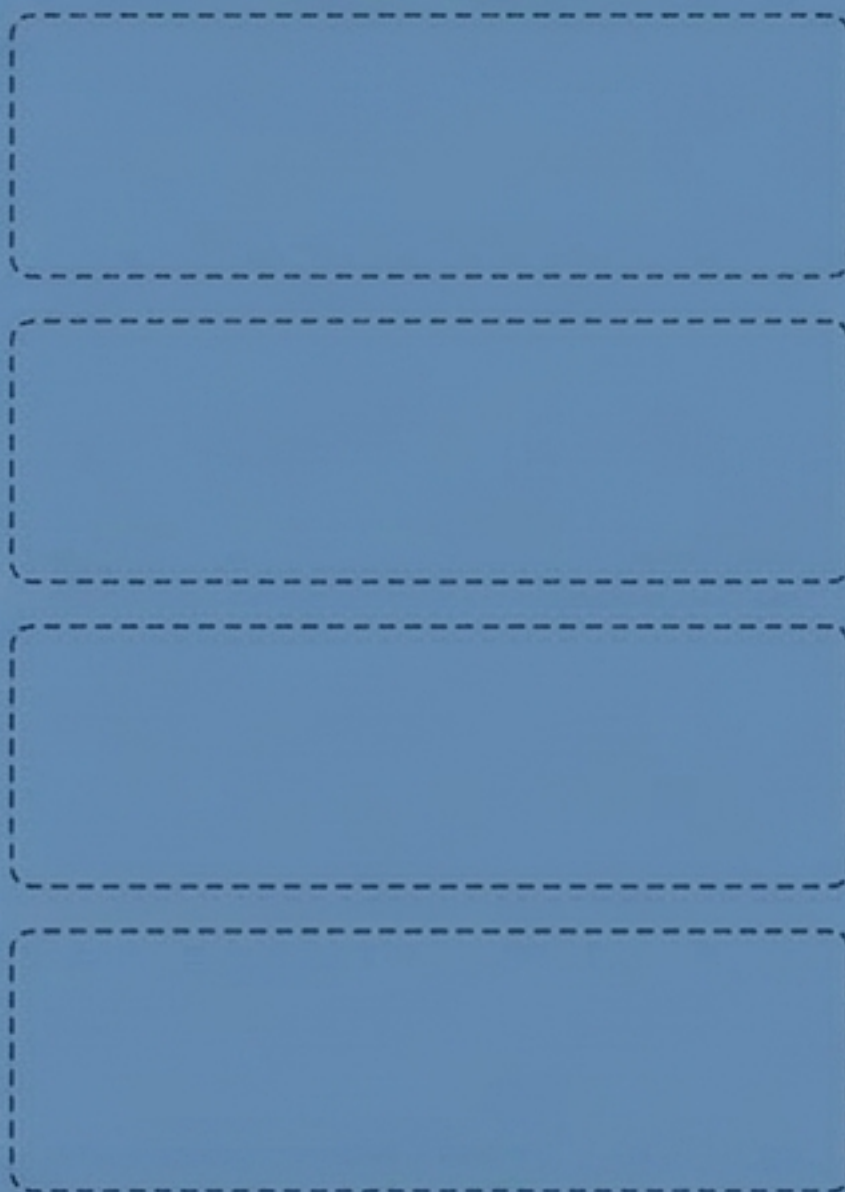
The UI Toolkit handles diverse multimodal data types

	<code>gr.Textbox</code>	General user queries, chatbot text input.	Strings
	<code>gr.File</code>	Document ingestion, uploading PDFs for knowledge base.	File paths/objects
	<code>gr.Slider</code>	Tuning model parameters (e.g., temperature, max tokens).	Integers/Floats (configured via min/max steps)
	<code>gr.Dropdown</code>	Forcing mutually exclusive selections (e.g., choosing an LLM model).	Lists/Arrays

Constructing the QA Bot: A 60-minute assembly

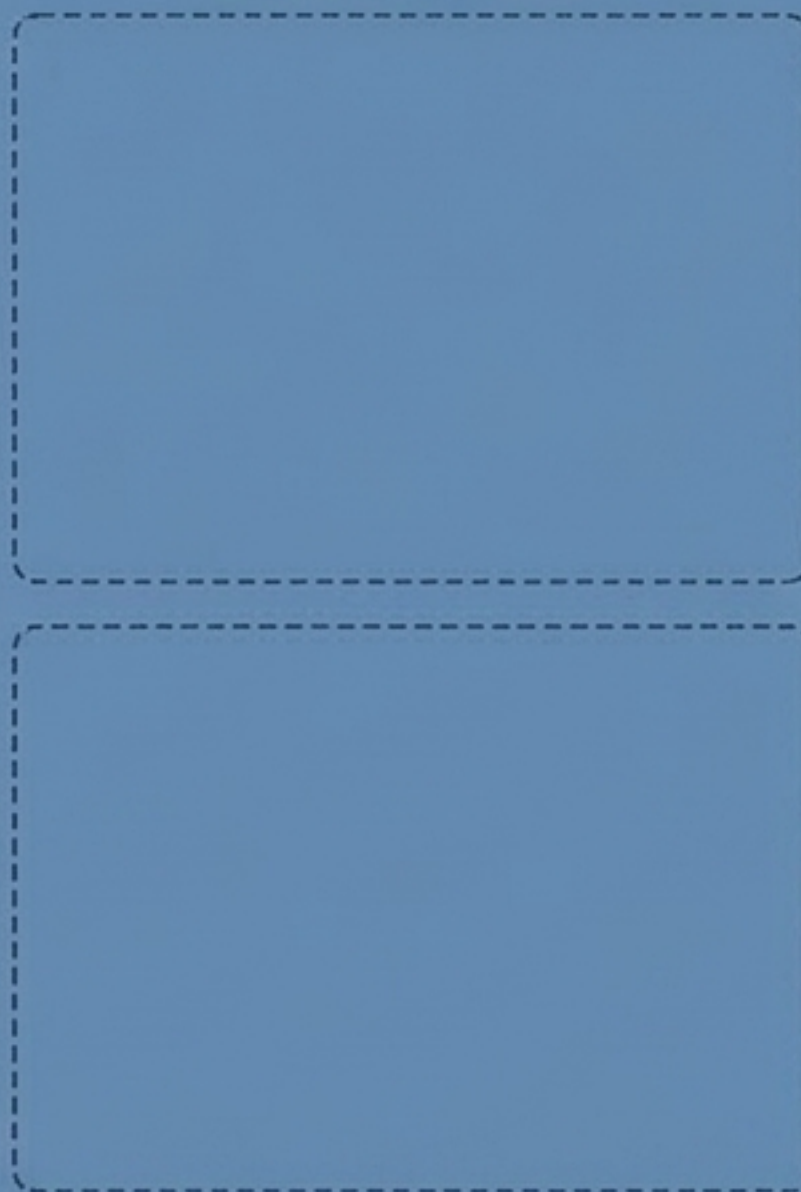
The Goal: Build an intelligent assistant that instantly answers queries based on loaded PDF manuals. Instead of writing complex infrastructure from scratch, we will orchestrate the application using LangChain, IBM Watsonx, and Gradio.

Data Pipeline



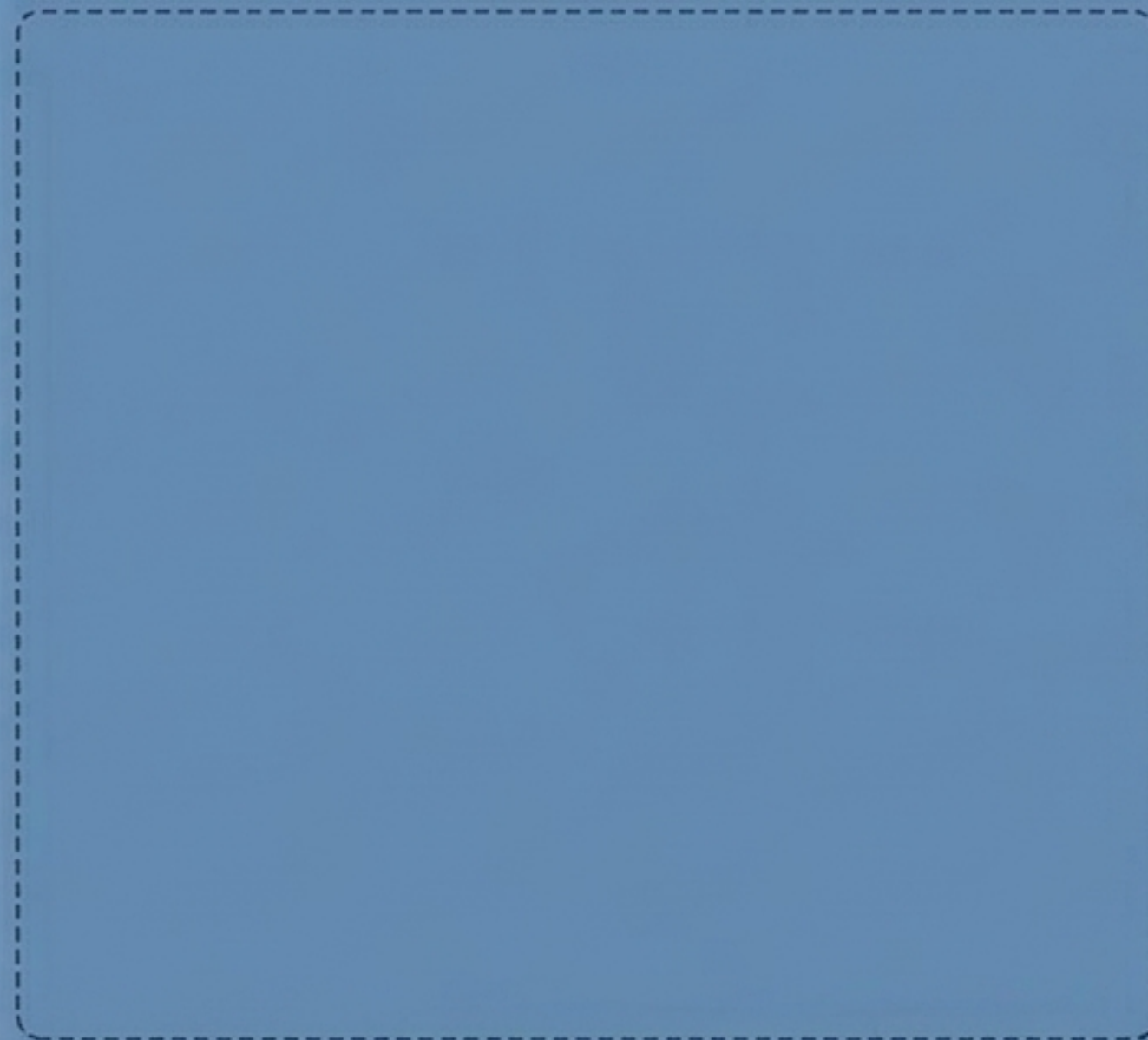
Four dashed rectangular boxes stacked vertically, representing steps in a data pipeline.

Intelligence



Two dashed rectangular boxes stacked vertically, representing components of intelligence.

Interface



A single large dashed rectangular box representing the interface.

Stacking the Backend: Document loading to vector storage

The Goal: Build an intelligent assistant that instantly answers queries based on loaded PDF manuals. Instead of writing complex infrastructure from scratch, we will orchestrate the application using LangChain, IBM Watsonx, and Gradio.

Data Pipeline

PyPDFLoader

Ingests the raw PDF file.



RecursiveCharacterTextSplitter

Fractures the document (Chunk Size: 1000) to preserve sentence context.



WatsonxEmbeddings

Applies the ibm/slate-125m model to generate semantic vectors.



ChromaDB

Initialized via `Chroma.from_documents` to store embedded chunks.

Intelligence

Interface

Stacking the Intelligence: Retrievers and LLMs

The Goal: Build an intelligent assistant that instantly answers queries based on loaded PDF manuals. Instead of writing complex infrastructure from scratch, we will orchestrate the application using LangChain, IBM Watsonx, and Gradio.

Data Pipeline

PyPDFLoader

Ingests the raw PDF file.



RecursiveCharacterTextSplitter

Fractures the document (Chunk Size: 1000) to preserve sentence context.



WatsonxEmbeddings

Applies the ibm/slate-125m model to generate semantic vectors.



ChromaDB

Initialized via `Chroma.from_documents` to store embedded chunks.

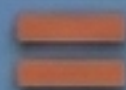
Intelligence

`ChromaDB.as_retriever()`



WatsonxLLM

Generative model (Mistral).
Max tokens: 256, Temp: 0.5.



`RetrievalQA.from_chain_type`

The LangChain orchestrator. Routes query to retriever, pulls chunks, passes context to the LLM.

Interface

Stacking the Frontend: The Gradio wrapper

The application is launched locally on port 7860, creating an instantly shareable web URL.

gr.Interface

Data Pipeline

PyPDFLoader
Ingests the raw PDF file.



RecursiveCharacterTextSplitter
Fractures the document (Chunk Size: 1000) to preserve sentence context.



WatsonxEmbeddings
Applies the ibm/slate-125m model to generate semantic vectors.



ChromaDB
Initialized via `Chroma.from_documents` to store embedded chunks.

Intelligence

`ChromaDB.as_retriever()`



WatsonxLLM
Generative model (Mistral).
Max tokens: 256, Temp: 0.5.



`RetrievalQA.from_chain_type`
The LangChain orchestrator. Routes query to retriever, pulls chunks, passes context to the LLM.

Interface

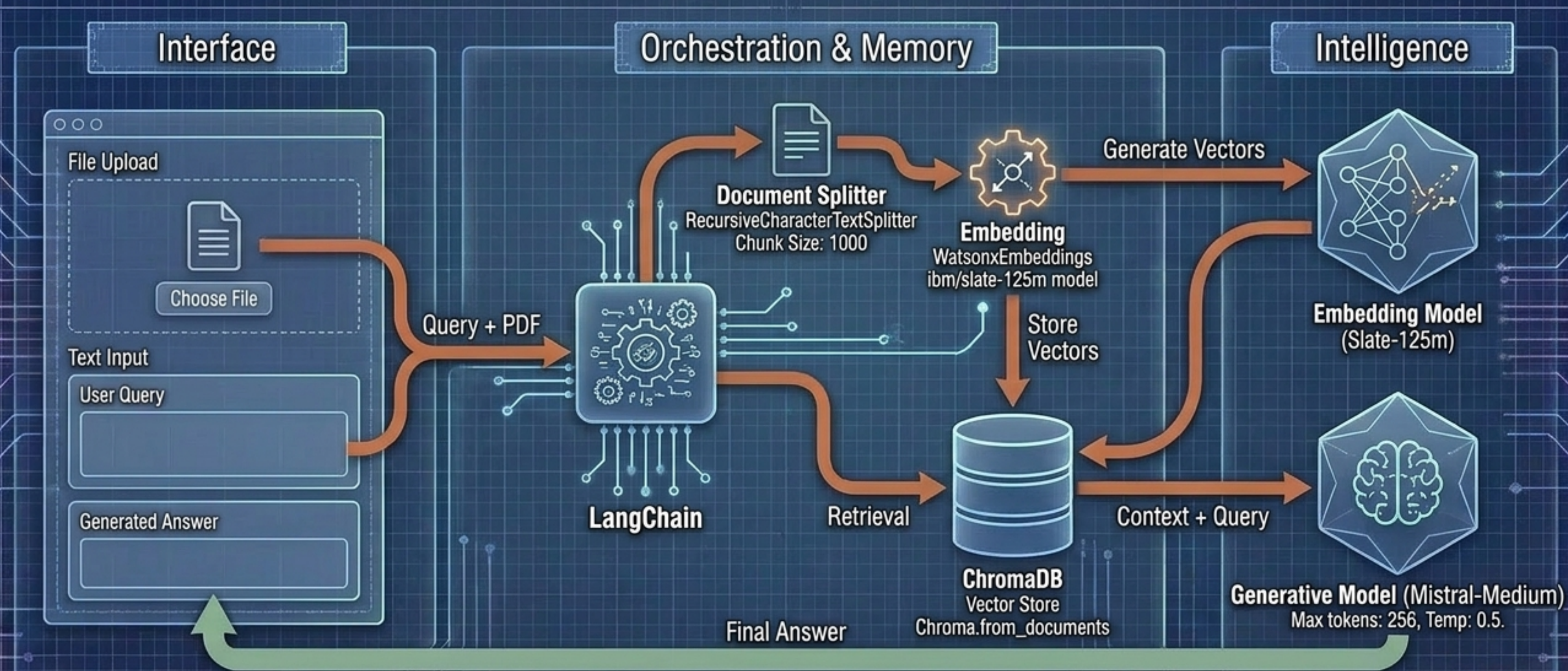
Inputs:
`gr.File`
(PDF upload)

Inputs:
`gr.Textbox`
(User query)

Outputs:
`gr.Textbox`
(Generated Answer)

The Full RAG Application Stack

The ecosystem in motion: The UI captures the intent, LangChain orchestrates the data pipeline, ChromaDB provides the memory, and Watsonx delivers the intelligence. Every module is isolated, swapable, and highly scalable.



Engineered for accuracy, deployed with ease

- Retrieval-Augmented Generation bridges the gap between general AI capabilities and specific, proprietary enterprise knowledge.
- By combining the memory of vector databases with the logic of LangChain and the accessibility of Gradio, developers can construct transparent, context-aware AI systems in minutes.
- The blueprint is complete. The next step is to build.

